

메신저 / 백엔드 인프라

정진우 (Jinwoo Jeong / チョンジウ)

메신저 Overview

- app/messenger/ - 메신저 백엔드 .
 - service/ 와 분리된 마이크로서비스 구조 .
 - WS/STOMP 를 사용하며 서로 다른 인프라를 쓰므로 분리
 - PostgreSQL Database 는 공유 , Table 은 백엔드 서비스별 소유 .
 - JDBCTemplate 으로 읽기는 함 . service/ 가 소유한 테이블에 Write 작업 x
 - In-memory 브로커가 아닌 RabbitMQ 가 broker. messenger/ 는 relay 만 수행 .
 - Spring Boot 앱은 relay 로 사용 . 확장 필요 시 messenger app 을 더 생성해 브로커와 연결 가능 .
- client/src/api/stomp.ts - STOMP 클라이언트
- client/src/api/messenger.ts – 히스토리 . Restful Request 처리 .

STOMP destination(messenger/ → client)

- /topic → broadcast
 - /topic/channel.{id}: 채널 메시지 CRUD
 - /topic/channel.{id}.read: 구독용 커서
 - /topic/workspace.{id}.presence: 온라인 상태 관리 (Redis)
- /user/queue → 개인별 큐
 - /user/queue/dm → DM 메시지
 - /user/queue/notifications → 푸시 알림
 - /user/queue/presence → Friend 접속 상태
 - /user/queue/errors → 에러

STOMP destination(client → messenger)

- /app/channel/{id}/send|update|delete|read: 채널 메시지 CRUD 및 기록 커서
- /app/dm/send: DM 보내기
- /app/dm/{id}/update|delete: dm 수정 / 삭제

messenger 백엔드 - STOMP 연결

- /ws 의 경우 CORS 는
 - Request : permitAll()(STOMP Connect 프레임에서 인증).
 - Origin: 5173(개발), Cloudflare(prod 환경)
- service/ 에서 issue 한 JWT 사용
- STOMP 클라이언트에서 보내는 subscribe 에 대해 membership 확인
- RabbitMQ 를 broker 로 사용하고 messenger/ 의 경우 relay 하는 역할을 함 . In-memory 브로커와 달리 messenger/ 인스턴스가 routing 을 안 하며 scalable 하게 만들기 쉬움 .
 - /vhost 의 경우 개발은 기본 ‘ / ’ 사용 . VPS 상에서는 별도 할당

* /ws 가 아닌 messenger.ts 를 통해 오는 RESTful Request 의 경우 JWT Filter 사용

messenger 백엔드 - Channel

- ChannelMessage 엔티티 / 테이블 기반 . messenger/ 소유 임 /topic/channel.{id} 로 broadcast.
- Channel 의 경우 membership 검사 후 subscribe
- service/ 에서 채널 삭제 시 HTTP Request 를 통해 연관된 메시지 삭제 . 이 때 조회를 위해 JDBC Template 으로 Channel 테이블 Read 함 . Write 작업 x. 추후 서비스 별 DB 가 분리 된다면 HTTP Request 방식 고려 중 .
- 기록 기능 - 프론트에서 보내 온 내용을 기반으로 upsert

messenger 백엔드 - DM

- DirectMessage 엔티티 / 테이블 기반 . messenger/ 소유임 .
/user/queue/dm 에 유저 별로 보냄 (/topic 과 달리 broadcast x).
- 둘의 대화이므로 기록 기능은 존재 x

서버간 통신 (Internal JWT 사용)

- 서버 간 통신에 다양한 방법이 고려됨 .
 - Unix Socket 을 사용한 파일 기반 방식
 - Overhead 가 적지만 같은 시스템 내에 없으면 통신 어려움 . Scalable x.
 - gRPC 를 이용한 통신
 - 인프라 셋업이 복잡하며 대규모 throughput 이 필요하지 않으면 이점 적음
- 승자 : JWT 를 사용한 Restful 방식
 - 클라이언트용과 다르게 서버 간 Secret 을 공유하며 Symmetric 한 방식 . 양쪽에서 HMAC 을 이용함
 - 현재는 service/ → messenger/ 방향 API 만 존재 . 이 경우 service/ 에서 issuer 를 hasi-service 로 잡고 JWT 발행 . messenger/ 에서는 확인만 함 .

Notification

- Notification 은 클라이언트에서 푸시 알림 형식으로 받아 보는 내용임 .
서버 간 HTTP Request 는 Internal JWT 사용 .
 1. service/ 에서 작업 후 messenger/ 에 HTTP Request (serviceToMessenger OpenAPI 문서 참고)
 2. /messenger 에서 Notification Table 에 저장 후 유저 별 /user/queue/notifications 로 전송 .
 - 히스토리의 경우 DM/ChannelMessage 와 같이 HTTP Request 로 불러 옴 .

STOMP 클라이언트

- stomp.ts
 - 최초에 messenger/ 백엔드의 /ws 로 Connection Upgrade 를 함 .
 - onConnect , teardown, subscription 등 STOMP 와 관련된 프론트엔드 측 로직을 모두 포함
- messenger.ts
 - messenger/ 서비스로의 HTTP Request 담당 . 히스토리 등 .

Presence 관리

- Redis 에 세션 ID 저장
 - TTL 60 초 .
 - subscribe 된 workspace 및 개인별 queue 로 보내져서 제한된 유저의 presence 만 확인 가능
 - 워크스페이스 멤버 (/topic/workspace.{id}.presence)
 - 친구 (/user/queue/presence)

백엔드 인프라

- ssh 로 OVHcloud 의 VPS 에 접속 . 모두 Docker 이미지로 작동함 (PostgreSQL, Redis, RabbitMQ, hasi-service, hasi-messenger)
 - 관리 및 유지보수가 쉬움
 - VPS 인프라의 관리 코드는 별도 Private 레포에 있음
- Reverse Proxy 는 Caddy 사용 . (api.hasi-collab 는 VPS 로 연결 , hasi-collab 는 Cloudflare Workers 의 프론트 엔드)
- PostgreSQL instance 는 VPS 내 공유 . 프로젝트 별 별도 Database 를 줌 .(hasi-collab 의 경우 db 명 hasi_collab)
- RabbitMQ 역시 하나의 Docker 컨테이너 공유 . 프로젝트 별 Vhost 를 나눔 .
- Redis 의 경우 프로젝트 별 분리가 DB/RabbitMQ 보다 어렵고 메모리 사용이 가벼워 (현재 약 15~20MB 수준) 프로젝트 별로 하나의 컨테이너를 할당 .

VPS 상에서 서비스 돌리기

- /srv/projects/ 아래 모든 tenant 서비스를 모아 둬
 - hasi-collab 의 경우 /srv/projects/hasi-collab
- Git main 에 올라온 내용을 pull 후 Docker 이미지 빌드 후 실행
 - `sudo docker compose -f docker-compose.prod.yml build < 서비스명 >`
 - Hasi-collab 의 경우 hasi-service, hasi-messenger 2 개 존재
 - `sudo docker compose -f docker-compose.prod.yml up -d`
 - 위에서 빌드한 이미지 기반으로 서비스를 시작 . hasi-redis 도 할당함 .
- `docker-compose.prod.yml` `.env.prod`, `application-prod.yml` 등은 개발용과 별도 . `.env.prod` 의 경우 git 에 올라가지 않고 VPS 상에서 직접 수정 .